

Draft-OPD: On-Policy Distillation for Speculative Draft Models

Haodi Lei^{1,2} Yafu Li^{2,4,†} Haoran Zhang^{1,2} Shunkai Zhang^{2,5} Qianjia Cheng^{2,6}
 Xiaoye Qu² Ganqu Cui² Bowen Zhou^{2,3} Ning Ding^{3,2,†} Yun Luo^{2,†} Yu Cheng^{4,2}

¹ Shanghai Jiao Tong University ² Shanghai AI Laboratory ³ Tsinghua University

⁴ The Chinese University of Hong Kong ⁵ Peking University ⁶ Zhejiang University

ABSTRACT

Speculative decoding accelerates large language model inference by pairing a target model with a lightweight draft model whose proposed tokens are verified in parallel. A common way to build draft models, like EAGLE3 or DFlash is supervised fine-tuning (SFT) on target-generated trajectories. However, we observe that SFT quickly plateaus: the draft model’s acceptance length on test data stops improving. The reason is an offline-to-inference mismatch: In SFT, the drafter learns from fixed target-generated trajectories, whereas during speculative decoding it is evaluated on blocks proposed under its own policy. This motivates on-policy distillation (OPD), where the target model supervises the drafter on draft-induced states. Yet OPD remains difficult for draft models, as they cannot reliably roll out complete sequences independently, whereas target-assisted generation makes the collected sequences follow the target distribution and thus eliminates the on-policy signal. We therefore propose Draft-OPD, which uses target-assisted rollout for stable continuations and replays drafting from the verification-exposed error positions. This allows the drafter to learn from target feedback on both accepted and rejected proposals, focusing training on the draft-induced errors that limit speculative acceptance. Experiments show that Draft-OPD achieves over $5\times$ lossless acceleration for thinking models across diverse tasks, improving over EAGLE-3 and DFlash by 23% and 13%.



Project Page



Code



Models

1 Introduction

Recent advances in large language models (LLMs) have enabled strong performance across reasoning, coding, and general assistant tasks, but their growing model sizes and longer generations substantially increase inference cost (Yang et al., 2025; DeepSeek-AI et al., 2025). Speculative decoding (SD) mitigates this cost by using a lightweight draft model to propose tokens that are verified in parallel by a larger target model, preserving the target model’s output distribution (Leviathan et al., 2023; Chen et al., 2023). In SD, the speedup depends heavily on the quality of the draft model: when the drafter closely matches the target model, longer token spans can be accepted, reducing the number of expensive target-model decoding steps. Recent mainstream methods, such as EAGLE-3 and DFlash, have achieved strong acceleration by training lightweight draft models on target-generated trajectories. (Li et al., 2025; Chen et al., 2026).

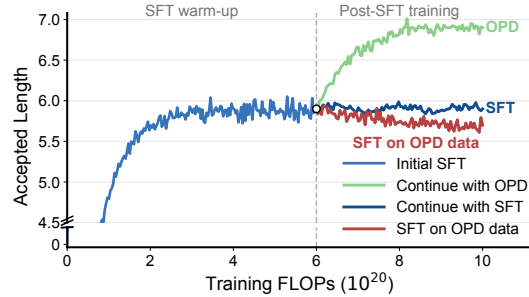


Figure 1: Accepted length during draft-model training. After an initial SFT warm-up, continued offline SFT quickly plateaus, and simply applying OPD data for SFT can even reduce accepted length. In contrast, Draft-OPD continues to improve accepted length by online training.

[†]Corresponding authors.

However, we find that offline supervised fine-tuning quickly reaches its limit for draft-model training. Figure 1 shows a representative training curve. After an initial SFT warm-up, continuing SFT does not improve the draft model’s acceptance length on test data; instead, the accepted length fluctuates around a fixed plateau. Moreover, continuing SFT on the data used for OPD can even reduce the accepted length. This suggests that the key limitation does not lie in offline training compute. Instead, the plateau points to a mismatch between the states used for training and the states that determine speculative acceptance. In SFT, the drafter learns from fixed target-generated trajectories, so every prefix is produced by the target model. During speculative decoding, however, the target model verifies token blocks proposed by the drafter itself. The accepted length is therefore determined by draft-induced inference states, rather than only by the static states seen in offline target trajectories.

This offline-to-inference mismatch motivates on-policy distillation (OPD), where the target model supervises states induced by the current draft policy (Agarwal et al., 2024). Directly applying OPD to draft models, however, is not straightforward. Standard OPD assumes that the student can roll out full sequences under its own policy. But mainstream draft models such as EAGLE-style or DFlash-style drafters are designed to propose short token spans under target-model guidance, rather than to act as standalone autoregressive generators. As a result, draft-only rollouts can easily become repetitive or low quality. Using target-assisted rollouts can produce usable sequences, but the verified continuation follows the target distribution: target model discards the incorrect tokens proposed by draft model, even though these errors are the most valuable training signals. Consequently, draft model still learns on trajectories generated by target model rather than on its own inference-time states.

To address this gap, we propose **Draft-OPD**, an on-policy distillation framework designed for speculative draft models. Draft-OPD uses target-assisted rollout to maintain stable continuations, and records error positions exposed by speculative verification. It then replays drafting from these positions and asks the target model to score the same draft-generated prefixes. This makes it possible to train on states where the draft model actually acted, including both accepted proposals and rejected proposals. Finally, Draft-OPD uses an acceptance-aware distillation objective that treats these two token groups differently: accepted tokens reinforce reliable agreement with the target model, while rejected tokens focus learning on draft-induced errors that limit speculative acceptance.

In summary, our contributions are:

- We identify a key limitation of offline SFT for draft models: SFT quickly plateaus because the drafter is trained on fixed target trajectories but evaluated on blocks induced by its own policy during speculative decoding.
- We explain why standard OPD does not directly apply to draft models: draft-only rollouts are unstable, while target-assisted rollouts remove the on-policy signal.
- We propose **Draft-OPD**, an on-policy distillation framework with error-position replay, making it feasible to efficiently post-train draft models on verification-time errors.
- Experiments show that Draft-OPD achieves over $5\times$ lossless acceleration for thinking models and improves over EAGLE-3 and DFlash by 23% and 13% under matched FLOPs.

2 Related Work

2.1 Speculative Decoding

Speculative decoding accelerates autoregressive inference by using a lightweight draft model to propose multiple future tokens and a larger target model to verify them in parallel while preserving the target distribution (Leviathan et al., 2023; Chen et al., 2023). Recent work further improves speculative decoding by leveraging feature-level context from the frozen target model, as in EAGLE (Li et al., 2024; 2025), or by designing stronger block-level draft architectures such as DFlash (Chen et al., 2026). These methods substantially improve decoding efficiency, but they still train draft models with offline SFT on target-generated trajectories, which limits further improvements in draft-model acceptance length. In contrast, our work studies how to move draft models beyond this offline training recipe.

2.2 Draft Model Distillation

Because speculative speedup depends on how closely the draft distribution matches the target distribution, several methods train draft models through distillation. DistillSpec aligns a compact draft model with the target model and highlights the importance of on-policy data and task-specific divergence choices (Zhou et al., 2024); temperature-centric distillation studies further show that matching training and decoding configurations can improve speculative decoding under challenging sampling settings (Ouyang et al., 2024). Online speculative decoding updates draft model from observed query distributions during deployment (Liu et al., 2023), while broader on-policy distillation trains students on self-generated sequences to reduce exposure bias (Agarwal et al., 2024). However, almost all previous work only studies standalone autoregressive draft models, which are often selected as smaller models from the same family as the target model. Our Draft-OPD is designed for training-based draft models such as DFlash or EAGLE3. These draft models cannot independently generate full student trajectories, making online distillation difficult because training must obtain usable rollouts rather than meaningless repetitions.

3 Preliminary

Speculative Decoding. Speculative decoding accelerates autoregressive generation by pairing a target model p_θ with a lightweight draft model q_ϕ (Leviathan et al., 2023; Chen et al., 2023). Given a prefix $x_{<t}$, the draft model proposes a block of K candidate tokens,

$$\begin{aligned} \hat{y}_{t+k} &\sim q_\phi(\cdot \mid x_{<t}, \hat{y}_{t:t+k-1}), \\ k &= 0, \dots, K-1. \end{aligned} \quad (1)$$

and the target model verifies these tokens in parallel. The verifier accepts the longest valid prefix of the drafted block and then continues generation from the verified prefix, preserving the target model’s output distribution while reducing the number of expensive target-model decoding steps.

Draft models can be categorized by how they are obtained. Model-based draft models use smaller autoregressive models from the same family as the target model (Miao et al., 2024). A more mainstream approach trains a lightweight draft model specifically for a given target model (Cai et al., 2024; Hui et al., 2026; Liu et al., 2026). These training-based draft models often share the target model’s embedding layer and LM head, and are designed to predict short draft spans rather than to generate complete sequences independently. In this work, we focus on training-based draft models because they can provide stronger acceleration.

A central metric for speculative decoding is the accepted length τ , i.e., the number of draft tokens accepted in each verification round. Higher τ means that each target-model verification generates more tokens, which directly improves decoding efficiency. It also reflects the alignment between the draft and target models, with better alignment yielding longer accepted spans.

On-Policy Distillation. On-policy distillation trains a student model on states induced by the student’s own policy, rather than only on fixed teacher-generated trajectories (Agarwal et al., 2024). Given a prompt x , the current student samples a trajectory and induces prefix states

$$\tilde{y} \sim q_\phi(\cdot \mid x), \quad s_t = (x, \tilde{y}_{<t}). \quad (2)$$

At each state s_t , the teacher provides the next-token distribution. Writing $p_\theta^t = p_\theta(\cdot \mid s_t)$ and $q_\phi^t = q_\phi(\cdot \mid s_t)$, a standard OPD objective is

$$\mathcal{L}_{\text{OPD}} = \mathbb{E}_{s_t} [D_{\text{KL}}(p_\theta^t \parallel q_\phi^t)]. \quad (3)$$

Thus, OPD directly targets the exposure mismatch between offline supervised training and inference-time generation: the student learns from states that it actually visits under its own policy.

This perspective is particularly relevant to speculative decoding. As discussed above, the accepted length τ is determined by how well the draft model aligns with the target model on the prefixes encountered during the draft-verify process. However, SFT trains q_ϕ only on fixed target-generated trajectories, whereas speculative decoding evaluates blocks produced by q_ϕ itself. OPD offers a natural way to reduce this mismatch by using the target model to supervise draft-induced states, particularly those induced by the draft model’s own deviations from the target.

4 Method

4.1 Challenges of Direct OPD

Applying OPD to draft models requires both stable rollouts and draft-induced training states. Standard OPD assumes that the student can roll out full sequences under its own policy, but EAGLE- and DFlash-style draft modules are designed to propose short token blocks under target-model verification, rather than to act as standalone autoregressive generators. As shown in Figure 2(a), forcing such draft modules to self-rollout full trajectories can produce repetitive or degenerate samples, making the resulting supervision unreliable.

Target-assisted rollout avoids degenerate samples, but it removes the draft-policy signal that OPD is meant to capture. As illustrated in Figure 2(b), strict speculative verification is lossless with respect to the target model, so the verified continuation follows the target distribution rather than the draft policy. Moreover, this brings back the offline-to-inference mismatch: target-assisted keeps accepted tokens and discards rejected proposals, even though these rejected tokens reveal the draft model’s most informative errors. We empirically verify this limitation in Section 5.3, where naive target-assisted rollout underperforms Draft-OPD. A suitable OPD method for draft modules must therefore keep target-quality rollouts while preserving the draft-induced errors exposed during verification.

4.2 Draft-OPD

Motivated by these failure modes, we design Draft-OPD to enable effective on-policy training for draft models. The key idea is to use target assistance to keep rollouts stable, while replaying draft-induced errors to preserve the on-policy training signal. Draft-OPD implements this idea through three coordinated designs: rollout with error-position collection, replay for log-probability computation, and an acceptance-aware distillation objective.

Rollout with error-position collection. Let p_θ denote the target model and q_ϕ denote a lightweight draft model initialized from supervised draft-model training. Given a prompt x , speculative decoding asks q_ϕ to propose a block of K tokens and uses p_θ to verify the block in parallel. Draft-OPD uses this interaction to collect a target-quality rollout while recording each draft block’s starting position as an anchor for later error replay. Let the verified rollout be $y = (y_0, \dots, y_{T-1})$. During rollout step m , the current verified prefix ends at position a_m , where $a_m = -1$ denotes the beginning of the sequence. The draft model proposes a block

$$d_m = (d_{m,1}, \dots, d_{m,K}) \sim q_\phi(\cdot \mid x, y_{\leq a_m}), \quad (4)$$

and the target model verifies the block. We record a_m as an anchor before moving to the next step. If the verifier accepts r_m tokens from the block, the next anchor is $a_m + r_m$. This process is repeated until the rollout reaches the maximum generation length or the target model emits an end-of-sequence token.

Anchors preserve the draft model’s local actions without requiring the draft model to generate an entire sequence alone. The final rollout remains a high-quality target-model sample, but each anchor identifies a state where the draft model actually proposed a block during inference. Drafting from any anchor requires the target-model hidden states of the preceding tokens; since the rollout has already computed these hidden states on the same sample, Draft-OPD can reuse them during subsequent replay.

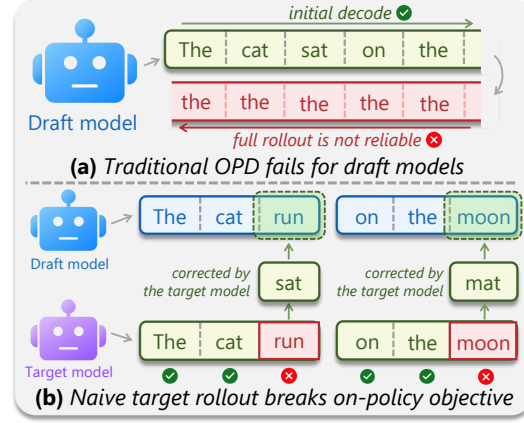


Figure 2: Why direct OPD is unsuitable for draft models. (a) Trajectories from draft model are repetitive. (b) Naive target-assisted rollout makes the sequence follow target distribution, losing informative errors from rejected tokens.

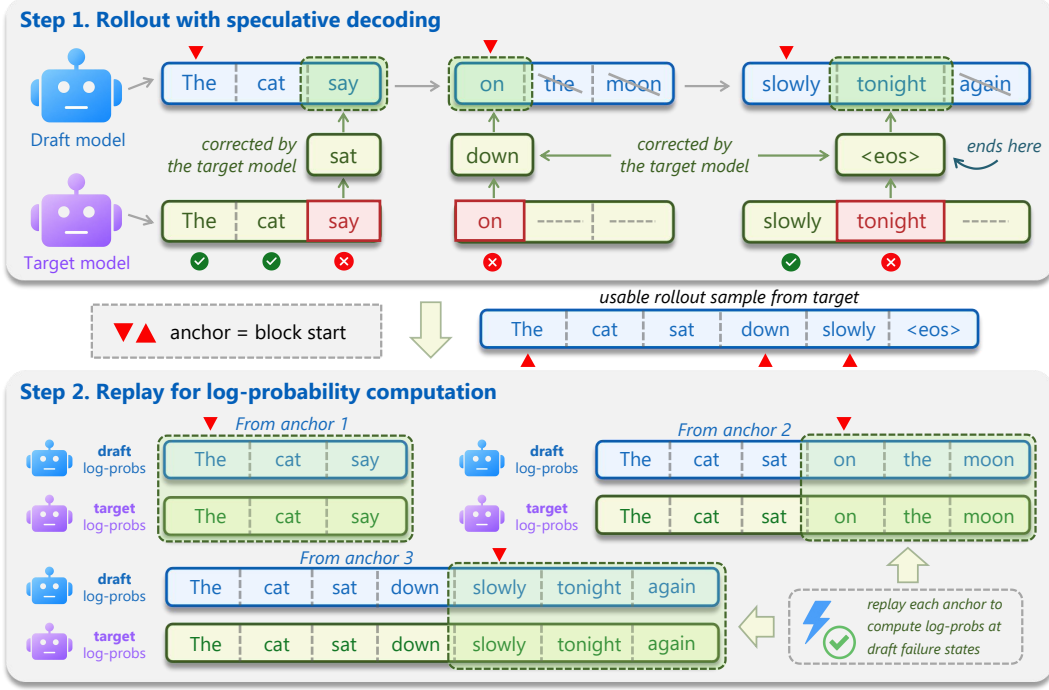


Figure 3: Draft-OPD for draft models. We use speculative decoding to collect stable rollouts and record the start index of each drafted block as an anchor. We then replay drafting from multiple anchors to compute student and teacher log-probabilities for both accepted and rejected draft tokens.

Replay for Log-Probability Computation. After collecting the rollout and anchors, Draft-OPD replays drafting from all anchors to compute the token-level student and teacher log-probabilities. For anchor a_m , define the replay context

$$c_m = (x, y_{\leq a_m}). \quad (5)$$

Starting from c_m , we replay the draft-generated block d_m . For each drafted token $d_{m,k}$, the draft model provides the student log-probability, while the target model provides the teacher log-probability on the same draft-generated prefix:

$$\log q_{m,k}(d_{m,k}) = \log q_\phi(d_{m,k} \mid c_m, d_{m,<k}), \quad (6)$$

$$\log p_{m,k}(d_{m,k}) = \log p_\theta(d_{m,k} \mid c_m, d_{m,<k}). \quad (7)$$

This replay step differs from training on the final rollout tokens: it evaluates the target model on the draft model’s proposed block, including positions that were rejected during verification.

The verification outcome naturally partitions drafted tokens into accepted and rejected sets. If r_m tokens are accepted from block m , then

$$\mathcal{I}_{\text{acc}} = \{(m, k) : 1 \leq k \leq r_m\}, \quad (8)$$

$$\mathcal{I}_{\text{rej}} = \{(m, k) : r_m < k \leq K\}. \quad (9)$$

Accepted tokens reflect states where the draft model agrees with the target well enough to pass verification. Rejected tokens include the first failed token and later positions in the same draft block, which become less reliable because an earlier rejection invalidates the remaining drafted suffix.

Acceptance-Aware Distillation Objective. We use different KL directions for accepted and rejected draft tokens. For accepted tokens, we use forward KL to make the draft distribution cover the target distribution at states where the draft model is already close to the target:

$$\mathcal{L}_{\text{acc}} = \frac{1}{|\mathcal{I}_{\text{acc}}|} \sum_{(m,k) \in \mathcal{I}_{\text{acc}}} D_{\text{KL}}(p_{m,k} \parallel q_{m,k}). \quad (10)$$

For rejected tokens, we use reverse KL to penalize the draft model’s own high-probability modes when the target model disagrees:

$$\mathcal{L}_{\text{rej}} = \frac{1}{Z} \sum_{(m,k) \in \mathcal{I}_{\text{rej}}} w_k D_{\text{KL}}(q_{m,k} \parallel p_{m,k}), \quad (11)$$

where $Z = \sum_{(m,k) \in \mathcal{I}_{\text{rej}}} w_k$ normalizes the rejected-token weights. We give the detailed rationale for this KL design in Appendix B.

Rejected tokens at later positions in a draft block are less important than earlier rejected tokens. In speculative decoding, an early error prevents the verifier from using the remaining suffix, so mistakes near the beginning of a block have a larger effect on acceptance length. We reflect this point with an exponentially decaying weight over block positions:

$$w_k = \gamma^{k-1} \quad (12)$$

The final Draft-OPD objective averages the accepted-token and rejected-token losses:

$$\mathcal{L}_{\text{Draft-OPD}} = \frac{\lambda_{\text{acc}} \mathcal{L}_{\text{acc}} + \lambda_{\text{rej}} \mathcal{L}_{\text{rej}}}{\lambda_{\text{acc}} + \lambda_{\text{rej}}}, \quad (13)$$

We set $\lambda_{\text{acc}} = \lambda_{\text{rej}} = 1$ in all experiments. This objective trains the draft model on stable target-assisted rollouts while preserving the draft-policy errors that determine speculative acceptance.

5 Experiment

Models and tasks. We conduct experiments on the Qwen3 family (Yang et al., 2025), including Qwen3-4B, Qwen3-8B, and Qwen3-30B-A3B-Thinking-2507. We evaluated three categories of benchmarks: mathematical reasoning, including GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023) and AIME (Mathematical Association of America, 2025); code generation and software engineering, including MBPP (Austin et al., 2021), HumanEval (Chen et al., 2021) and SWE-bench Lite (Jimenez et al., 2024); and out-of-domain benchmark MT-Bench (Zheng et al., 2023).

Datasets. For the SFT stage, we use the same training data mixture as DFlash (Chen et al., 2026). For the OPD stage, we construct a 16K sample prompt pool randomly sampling 2K prompts from the GSM8K training set, 5K prompts from the MATH corpus after excluding MATH-500 held-out examples, 4K prompts from AoPS (Art of Problem Solving, 2026), and 5K prompts from CodeAlpaca (Chaudhary, 2023). We use only the questions or instructions from these datasets; responses are generated online by the target model during OPD rather than taken from static reference answers.

Implementation. Unless otherwise specified, all experiments are conducted on NVIDIA H200 GPUs, with a batch size of 1, thinking mode enabled. We perform Draft-OPD on top of SFT-trained DFlash draft models: the draft models use 5 Transformer layers for Qwen3-4B and Qwen3-8B, and 8 layers for Qwen3-30B-A3B-Thinking (Chen et al., 2026). We use a block size of 16 for both training and inference. Detailed hyperparameters for OPD training are provided in Appendix A.

Baselines. We compare against EAGLE-3 (Li et al., 2025) and DFlash (Chen et al., 2026). For a fair comparison, both EAGLE-3 and DFlash are trained with the data mixture introduced by DFlash, and their SFT training budget is matched to the total SFT plus OPD budget of our method so that all draft models are trained under approximately the same FLOPs budget.

Metrics. We focus on efficiency-related metrics and do not report generation quality, since Draft-OPD only post-trains the draft model and does not change the speculative decoding procedure used at inference time, thereby preserving the exact output distribution of the target model.

- **Speedup Ratio.** The actual test speedup ratio relative to vanilla autoregressive decoding.
- **Average acceptance length (τ).** The average number of draft tokens accepted by the target model in each verification cycle.

Model	Method	Speedup							Mean	
		MATH			CODE			CHAT	Speedup	τ
		GSM8K	MATH-500	AIME25	MBPP	HumanEval	SWE-Lite	MT-Bench		
Thinking Mode Enabled										
Temperature = 0										
Q3-4B	EAGLE-3	4.41×	4.15×	3.30×	4.29×	4.38×	3.47×	3.07×	3.87×	5.33
	DFlash	4.51×	4.88×	4.69×	4.39×	4.77×	4.12×	2.96×	4.33×	5.51
	Draft-OPD	5.31×	5.55×	5.28×	4.85×	5.17×	4.66×	3.18×	4.86×	5.96
Q3-8B	EAGLE-3	4.58×	4.45×	4.01×	4.55×	4.46×	3.38×	3.02×	4.06×	5.64
	DFlash	4.67×	5.01×	4.77×	4.50×	4.71×	3.72×	3.01×	4.34×	5.19
	Draft-OPD	5.36×	5.80×	5.51×	4.86×	5.19×	4.31×	3.18×	4.89×	5.73
Temperature = 0.6										
Q3-4B	EAGLE-3	3.90×	3.77×	3.07×	3.75×	3.77×	2.75×	2.77×	3.40×	5.03
	DFlash	4.21×	4.46×	4.25×	3.92×	4.17×	3.17×	2.73×	3.84×	4.77
	Draft-OPD	4.73×	4.91×	4.56×	4.21×	4.40×	3.38×	2.87×	4.15×	5.13
Q3-8B	EAGLE-3	4.13×	4.09×	3.57×	3.93×	3.95×	2.83×	2.83×	3.62×	5.33
	DFlash	4.23×	4.57×	4.34×	3.85×	4.12×	2.88×	2.76×	3.82×	4.66
	Draft-OPD	4.77×	5.07×	4.80×	4.17×	4.47×	3.13×	2.91×	4.19×	5.05
Thinking Mode Disabled										
Temperature = 0										
Q3-4B	EAGLE-3	4.58×	5.70×	5.39×	4.55×	4.53×	2.54×	2.80×	4.30×	5.84
	DFlash	5.36×	6.35×	5.93×	5.00×	5.19×	3.05×	3.01×	4.84×	6.04
	Draft-OPD	6.22×	7.22×	6.39×	5.40×	5.65×	3.18×	3.09×	5.31×	6.60
Q3-8B	EAGLE-3	4.99×	6.07×	5.87×	4.83×	4.94×	2.67×	3.06×	4.63×	5.99
	DFlash	5.69×	6.81×	6.40×	5.17×	5.64×	3.17×	2.92×	5.11×	6.04
	Draft-OPD	6.49×	7.64×	6.99×	5.64×	6.02×	3.33×	3.12×	5.60×	6.57
Temperature = 0.6										
Q3-4B	EAGLE-3	4.16×	4.93×	4.49×	3.98×	4.02×	2.41×	2.50×	3.78×	5.66
	DFlash	5.04×	5.87×	4.99×	4.72×	5.17×	2.77×	2.80×	4.48×	5.73
	Draft-OPD	5.84×	6.35×	5.31×	4.98×	5.42×	2.90×	2.86×	4.81×	6.13
Q3-8B	EAGLE-3	4.61×	5.17×	4.85×	4.40×	4.29×	2.43×	2.70×	4.06×	5.69
	DFlash	5.26×	6.18×	5.19×	4.74×	4.97×	2.85×	2.81×	4.57×	5.62
	Draft-OPD	6.01×	6.61×	5.57×	5.17×	5.30×	3.03×	2.97×	4.95×	6.02

Table 1: Decoding speedup ratio and average acceptance length (τ) on Qwen3 models. For thinking mode, we use a maximum of 8192 generated tokens; for non-thinking mode, we use a maximum of 2048 generated tokens.

5.1 Main Results

Table 1 compares Draft-OPD with EAGLE-3 and DFlash on Qwen3 models under a matched training FLOPs budget. For EAGLE-3, we use a tree size of 16, with draft steps and top- k set to 8 and 4, respectively. We evaluate both greedy decoding and the recommended Qwen reasoning sampling setting, with temperature 0.6, top- p 0.95, and top- k 20.

Thinking Mode Enabled. With thinking mode enabled, Draft-OPD consistently improves acceptance length and decoding speed under matched training FLOPs. At temperature 0, it raises the two-model average τ from 5.35 for DFlash to 5.85 and achieves 4.88× average speedup across the seven benchmarks, improving over EAGLE-3 and DFlash by 23% and 13%.

At temperature 0.6, it remains the fastest method, averaging 4.17× speedup. Although EAGLE-3 obtains the highest τ on Qwen3-8B at temperature 0.6, its sequential drafting limits wall-clock speed, supporting our choice to apply OPD to DFlash-style parallel drafting.

Thinking Mode Disabled. With thinking mode disabled, Draft-OPD preserves the same advantage across decoding temperatures and model sizes. It maintains an average acceptance length of 6.33 and achieves a 5.17× average speedup. Together with the thinking-mode results, this shows that Draft-OPD improves draft-target alignment across both long reasoning traces and shorter non-thinking generations.

Performance on SGLang. We benchmark Draft-OPD and DFlash on SGLang (Zheng et al., 2024) with the FA3 backend to evaluate deployment-time efficiency. Table 2 reports serving throughput under concurrency levels from 1 to 32, together with the average acceptance length τ .

Draft-OPD consistently improves SGLang serving efficiency across all evaluated models, tasks, and concurrency levels. It improves the acceptance length by 11.2% on average over the evaluated model-task pairs and achieves up to a 17% speedup on Qwen3-30B-A3B-Thinking. Notably, the throughput gains do not diminish under higher concurrency: at concurrency 32, the average relative gain is even higher than at concurrency 1. These serving results show that the higher acceptance lengths produced by Draft-OPD translate into practical throughput gains in an optimized inference engine.

5.2 Ablation Study

Training Data. A key question is whether the gains of Draft-OPD come from on-policy distillation or simply from exposing the draft model to more prompts. Figure 4 compares Draft-OPD with EAGLE-3 and DFlash variants that continue supervised training on responses generated by the target model from the OPD prompt pool. Draft-OPD performs better, showing that its improvement is not explained by additional supervised data alone. Instead, the benefit comes from distilling target distributions on states induced by the draft model during speculative decoding.

KL Type. The acceptance-aware KL objective is designed to match the asymmetric roles of accepted and rejected draft tokens. Table 3 studies this design by replacing the mixed objective with all-forward and all-reverse KL variants. The all-forward variant is consistently below Draft-OPD, while the all-reverse variant performs the worst among the KL variants. This supports using different KL directions for accepted and rejected positions rather than applying a single divergence to all draft tokens.

Anchor Position. Error-position replay is intended to focus training on the states where speculative decoding actually fails. Table 3 compares this design with randomly selected replay anchors. Random anchors lead to lower speedup and acceptance length, indicating that not all on-policy states are equally useful for draft-model post-training. Concentrating replay around rejected proposals better exposes the errors that determine speculative acceptance.

Task	Method	Concurrency					Avg. τ
		1	4	8	16	32	
Qwen3-4B (Enable Thinking)							
AIME25	DFlash	912	2755	4750	6841	8410	6.06
	Draft-OPD	969 (6% $\uparrow$)	2984 (8% $\uparrow$)	5071 (7% $\uparrow$)	7297 (7% $\uparrow$)	9043 (8% $\uparrow$)	6.59
MATH-500	DFlash	949	3157	5234	8004	10062	6.17
	Draft-OPD	1036 (9% $\uparrow$)	3413 (8% $\uparrow$)	5603 (7% $\uparrow$)	8593 (7% $\uparrow$)	10943 (9% $\uparrow$)	6.68
SWE-Lite	DFlash	901	2983	5009	7743	9604	5.62
	Draft-OPD	976 (8% $\uparrow$)	3230 (8% $\uparrow$)	5544 (11% $\uparrow$)	8568 (11% $\uparrow$)	10538 (10% $\uparrow$)	6.12
Qwen3-8B (Enable Thinking)							
AIME25	DFlash	662	2121	3612	4956	5985	5.67
	Draft-OPD	741 (12% $\uparrow$)	2465 (16% $\uparrow$)	4127 (14% $\uparrow$)	5729 (16% $\uparrow$)	6645 (11% $\uparrow$)	6.42
MATH-500	DFlash	703	2374	4154	5958	6991	5.99
	Draft-OPD	787 (12% $\uparrow$)	2721 (14% $\uparrow$)	4691 (13% $\uparrow$)	6666 (12% $\uparrow$)	7940 (13% $\uparrow$)	6.64
SWE-Lite	DFlash	592	1962	3347	5051	6113	4.60
	Draft-OPD	644 (9% $\uparrow$)	2263 (15% $\uparrow$)	3879 (15% $\uparrow$)	5611 (11% $\uparrow$)	6904 (13% $\uparrow$)	5.27
Qwen3-30B-A3B-Thinking-2507							
AIME25	DFlash	421	1086	1858	2738	4014	4.54
	Draft-OPD	476 (13% $\uparrow$)	1229 (13% $\uparrow$)	2111 (13% $\uparrow$)	3187 (16% $\uparrow$)	4718 (17% $\uparrow$)	5.32
MATH-500	DFlash	417	1176	2009	3020	4462	5.44
	Draft-OPD	477 (14% $\uparrow$)	1303 (11% $\uparrow$)	2243 (11% $\uparrow$)	3405 (12% $\uparrow$)	5007 (12% $\uparrow$)	5.95
SWE-Lite	DFlash	319	855	1484	2337	3453	3.43
	Draft-OPD	352 (10% $\uparrow$)	960 (11% $\uparrow$)	1628 (9% $\uparrow$)	2579 (10% $\uparrow$)	3850 (11% $\uparrow$)	3.81

Table 2: Throughput (tok/s), speedup over DFlash, and average acceptance length (τ) on SGLang.

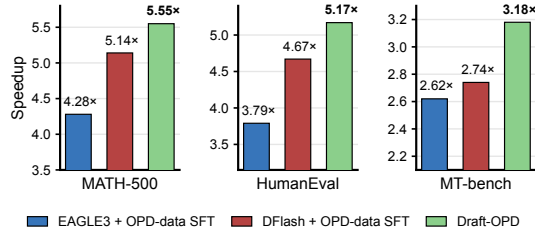


Figure 4: Training-data ablation on Qwen3-4B with thinking mode enabled.

Method	MATH-500		HumanEval		MT-Bench	
	Speedup	τ	Speedup	τ	Speedup	τ
Draft-OPD	5.55$\times$	6.57	5.17$\times$	6.18	3.18$\times$	4.44
w/o Weight Decay	5.13 $\times$	6.18	4.96 $\times$	6.01	3.07 $\times$	4.29
All-reverse KL	5.11 $\times$	6.14	4.94 $\times$	5.98	3.08 $\times$	4.33
All-forward KL	5.34 $\times$	6.35	5.01 $\times$	6.09	3.09 $\times$	4.33
Random Anchors	5.04 $\times$	6.08	4.99 $\times$	6.07	2.96 $\times$	4.24

Table 3: Component ablation on Qwen3-4B.

Weight Decay. The rejected-token position decay down-weights later rejected tokens in the same drafted block, since they are more likely to be affected by earlier draft errors. As shown in Table 3, removing this decay reduces both speedup and acceptance length. This suggests that the rejected-token loss is most effective when it emphasizes the earliest and most informative failure positions. We further analyze the robustness of Draft-OPD to the decay factor γ and relative rejected-token weight λ_{rej} in Appendix C.

5.3 Analysis Experiments

Target-Assisted Rollout. The direct draft-only rollout in Figure 2(a) quickly becomes repetitive and low quality, making it unsuitable for OPD training despite being on-policy. We therefore analyze the naive target-assisted rollout baseline illustrated in Figure 2(b).

Since speculative verification keeps the target-verified continuation and discards rejected draft proposals, this baseline reduces to KL-loss SFT on target-distributed trajectories. As shown in Table 4, naive target-assisted rollout underperforms Draft-OPD, reducing the average speedup from $4.63\times$ to $4.29\times$, a relative drop of 7.3%. This confirms that stable target-assisted rollouts alone are insufficient; preserving draft-induced errors is important for effective online draft-model training.

Setting	MATH-500		HumanEval		MT-Bench	
	Speedup	τ	Speedup	τ	Speedup	τ
Draft-OPD	$5.55\times$	6.57	$5.17\times$	6.18	$3.18\times$	4.44
Naive rollout	$5.06\times$	6.07	$4.80\times$	5.93	$3.02\times$	4.25

Table 4: Analysis of naive target-assisted rollout on Qwen3-4B.

Thinking-Mode Draftability Gap. We also observe that in Table 1, on math and code tasks, draft models trained under thinking mode are substantially less effective than their non-thinking counterparts. We attribute this gap to the higher uncertainty of target-model responses in thinking mode, where long reasoning traces often allow multiple plausible next steps, making it more difficult for the draft model to fit the target model’s distribution. Appendix E provides a slightly more detailed analysis.

6 Discussion

We discuss two alternatives to the acceptance-aware objective: restricting supervision to the first rejected token and extending positional decay to accepted tokens.

6.1 Training Only on the First Rejected Token

Draft-OPD applies the rejected-token objective to the suffix beginning at the first rejected token. Positions after the first rejection can be less reliable because they are conditioned on a draft prefix that has already been invalidated by the verifier. To assess whether these later positions provide useful supervision, we construct a *First-rejected-only* variant that keeps the accepted-token objective unchanged but applies the rejected-token loss only to the first rejected token in each failed block.

Benchmark	First-rejected-only	Draft-OPD
GSM8K	6.32	6.39
AIME25	6.36	6.32
MATH-500	6.57	6.57
MBPP	5.81	5.73
HumanEval	6.14	6.12
MT-Bench	4.32	4.44
SWE-Lite	6.08	6.07
Average	5.94	5.95

Table 5: Average acceptance length τ when applying the rejected-token loss only to the first rejected token.

As shown in Table 5, First-rejected-only and Draft-OPD achieve nearly identical average acceptance lengths of 5.94 and 5.95, respectively. This result suggests that the first rejected token captures most of the useful error signal, whereas later suffix tokens provide limited additional benefit and may introduce noise. Moreover, this objective is compatible with a truncated replay implementation that stops after scoring the first failure, potentially reducing replay-stage computation. First-rejected-only is therefore a promising efficiency-oriented simplification of Draft-OPD.

6.2 Why Decay Is Applied Only to Rejected Tokens

The positional decay in Draft-OPD is intended to account for the decreasing reliability of rejected suffixes, rather than to impose a general preference for earlier block positions. To test whether accepted tokens also benefit from positional decay, we construct an *Accept-decay* variant that applies the same decay factor $\gamma = 0.8$ to the accepted-token objective while keeping all other settings unchanged.

Benchmark	Accept-decay	Draft-OPD
GSM8K	6.25	6.39
AIME25	6.28	6.32
MATH-500	6.44	6.57
MBPP	5.66	5.73
HumanEval	6.11	6.12
MT-Bench	4.33	4.44
SWE-Lite	5.97	6.07
Average	5.86	5.95

Table 6: Average acceptance length τ when positional decay is also applied to accepted tokens.

Table 6 shows that Accept-decay reduces the acceptance length on all seven benchmarks, lowering the average from 5.95 to 5.86. Accepted tokens remain on the verified target trajectory and therefore provide reliable supervision throughout a block. Applying decay to positions later within the accepted portion of the block unnecessarily down-weights this reliable supervision, explaining why decay is beneficial for rejected suffixes but not for accepted tokens.

7 Conclusion

We propose Draft-OPD, an on-policy distillation framework for training-based draft models. By using target-assisted rollouts with an error-position replay mechanism, Draft-OPD keeps training samples stable while preserving the draft-policy errors that determine speculative acceptance. We further introduce an acceptance-aware distillation objective that treats accepted and rejected draft tokens differently, enabling the draft model to learn from both reliable proposals and informative failure modes. Experiments on Qwen3 models show that Draft-OPD improves average acceptance length and end-to-end decoding speedup over strong draft-model baselines under a matched training budget, with additional SGLang results confirming practical serving gains. Overall, Draft-OPD shows that draft models benefit from post-training on verification-time errors, offering an effective direction for improving training-based draft model.

Limitations

Training Length. Due to compute constraints, for thinking-mode Draft-OPD training we cap the maximum response length at 4096 tokens, while evaluation uses 8192 tokens. Although Draft-OPD still achieves strong gains under this longer evaluation setting, the training rollouts may not fully cover late-stage states in very long generations. Scaling OPD training to longer rollouts could expose the draft model to a broader range of verification-time errors and may further improve draft-target alignment for long outputs.

Evaluation Scope. This work focuses on post-training draft models for lossless speculative decoding. Our main experiments are conducted on Qwen3 models, and the Draft-OPD implementation

follows the DFlash-style parallel draft architecture. Although we include comparisons with EAGLE-3 and evaluate deployment in SGLang, further evaluation is needed to understand how well Draft-OPD transfers to other model families, draft architectures, and inference backends.

Lossless Decoding. Because speculative decoding preserves the target model distribution, Draft-OPD is designed to improve decoding efficiency rather than generation quality; extending on-policy draft-model training to settings with approximate or lossy verification is an interesting direction for future work.

Acknowledgments

This work was supported by the Shanghai Artificial Intelligence Laboratory. We are grateful to the authors and open-source communities whose work made this project possible. In particular, DFlash (Chen et al., 2026) and EAGLE-3 (Li et al., 2025) provided strong foundations and reference points for training-based speculative decoding, while SGLang (Zheng et al., 2024), SpecForge (Li et al., 2026), and verl (Sheng et al., 2024) offered practical infrastructure for serving and training experiments. We especially appreciate SpecForge for its open-source implementation of DFlash training, which provided a useful reference for our training setup. We also thank Runzhe Zhan for sharing valuable experience on OPD training and for discussions that helped us stabilize the training workflow.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/5be69a584901a26c521c2b51e40a4c20-Abstract-Conference.html.
- Art of Problem Solving. Art of problem solving, 2026. URL <https://artofproblemsolving.com>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. URL <https://arxiv.org/abs/2401.10774>.
- Sahil Chaudhary. Code alpaca: An instruction-following llama model for code generation. <https://github.com/sahil280114/codealpaca>, 2023.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.

- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shiron Ma, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *Nature*, 645:633–638, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Mude Hui, Xin Huang, Jaime Campos Salas, Yue Sun, Nathan Pemberton, Xiang Song, Ashish Khetan, and George Karypis. P-eagle: Parallel-drafting eagle with scalable training. *arXiv preprint arXiv:2602.01469*, 2026. URL <https://arxiv.org/abs/2602.01469>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang, Fan Yin, Shuai Shi, Yefei Chen, Xiaomin Dong, Qiaoling Chen, Jin Pan, Ji Li, Laixin Xie, Yineng Zhang, Lei Yu, Yonggang Wen, Ivor Tsang, and Tianwei Zhang. Specforge: A flexible and efficient open-source training framework for speculative decoding. *arXiv preprint arXiv:2603.18567*, 2026. URL <https://arxiv.org/abs/2603.18567>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. URL <https://arxiv.org/abs/2401.15077>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. URL <https://arxiv.org/abs/2503.01840>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyang Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*, 2026. URL <https://arxiv.org/abs/2601.19278>.
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2023. URL <https://arxiv.org/abs/2310.07177>.
- Mathematical Association of America. American invitational mathematics examination (aime) 2025, 2025. URL <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024. doi: 10.1145/3620666.3651335. URL <https://arxiv.org/abs/2305.09781>.

- D. Nathawani, S. Ding, V. Lavrukhin, I. Gitman, S. Majumdar, E. Bakhturina, B. Ginsburg, and J. Polak Scowcroft. Nemotron-Post-Training-Dataset-v2, aug 2025. URL <https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v2>.
- Siru Ouyang, Shuohang Wang, Minhao Jiang, Ming Zhong, Donghan Yu, Jiawei Han, and Yelong Shen. Temperature-centric investigation of speculative decoding with knowledge distillation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 13125–13137, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.767. URL <https://aclanthology.org/2024.findings-emnlp.767/>.
- ShareGPT. Sharegpt dataset. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2023.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024. URL <https://arxiv.org/abs/2409.19256>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2312.07104. URL <https://arxiv.org/abs/2312.07104>.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2024. URL <https://arxiv.org/abs/2310.08461>.

A Training Details

SFT stage. We follow the SFT configuration of DFlash (Chen et al., 2026) for most training settings and use SpecForge (Li et al., 2026) as the training framework. For Draft-OPD, we initialize the OPD stage from the draft-model checkpoint after 6 SFT epochs. For the EAGLE-3 and DFlash baselines used in our comparisons, we continue supervised draft-model training for 10 epochs under the same data setup and report the checkpoint with the best evaluation performance.

OPD stage. For Draft-OPD training, we use the rejected-token position weights

$$w_k = \gamma^{k-1},$$

with $\gamma = 0.8$. We train on the OPD data mixture described in Section 5 for 8 epochs, using a maximum response length of 4096 tokens for thinking-enabled and 2048 tokens for thinking-disabled. Optimization uses AdamW with a learning rate of 3×10^{-4} and a cosine learning-rate schedule with a warmup ratio of 0.05. We implement the OPD stage with verl (Sheng et al., 2024). For the final Draft-OPD objective in Equation 13, we set $\lambda_{\text{acc}} = \lambda_{\text{rej}} = 1$ in all experiments.

Training cost and reproducibility. All Draft-OPD experiments use NVIDIA H200 GPUs. The Qwen3-4B and Qwen3-8B experiments use 8 GPUs, while Qwen3-30B-A3B-Thinking uses 16 GPUs. For Qwen3-4B, the OPD stage takes approximately 15 hours on 8 H200 GPUs, corresponding to approximately 120 H200 GPU-hours. This accounting includes the frozen target model’s rollout and verification computation. Table 7 summarizes the Qwen3-4B OPD configuration.

Table 7: Training cost and configuration of the Qwen3-4B OPD stage.

Item	Qwen3-4B OPD setting
GPU hardware	8× NVIDIA H200
OPD wall-clock time	~15 hours
OPD GPU-hours	~120 H200 GPU-hours
Prompt pool	16K prompts, disjoint from SFT data
Global training batch size	24
OPD steps	~5.3K
Epochs / prompt reuse	8 epochs / 8 passes per prompt
Maximum rollout length	4096 (thinking), 2048 (non-thinking)

B Loss Design for Accepted and Rejected Draft Tokens

We provide a local training-objective justification for using different KL directions on accepted and rejected replay positions. The goal is not to show that the mixed KL objective universally dominates all-forward or all-reverse KL. For a fixed replay state and an unconstrained draft distribution, these objectives share the same optimum $q = p$. Their difference lies in how the local loss weights target-supported tokens and draft-proposed tokens during finite-capacity optimization.

Fix a replay state s and let $p(\cdot | s)$ and $q(\cdot | s)$ denote the target and draft next-token distributions over vocabulary $\mathcal{V}$. In the following derivation, all probabilities are conditioned on s , so we write $p(y)$ and $q(y)$ for brevity. For an accepted replay position, the draft proposal has passed target verification, and the local objective should match the target distribution at this reliable state. This gives the target-weighted cross-entropy

$$\mathcal{J}_{\text{acc}}(q) = \mathbb{E}_{y \sim p}[-\log q(y)]. \quad (14)$$

Expanding the expectation yields

$$\mathcal{J}_{\text{acc}}(q) = - \sum_{y \in \mathcal{V}} p(y) \log q(y) \quad (15)$$

$$= H(p) + D_{\text{KL}}(p \| q), \quad (16)$$

where $H(p)$ is independent of q . Thus, the accepted-position objective is equivalent, up to an additive constant, to minimizing forward KL.

For a rejected replay position, the relevant signal is draft-induced: the token belongs to a draft-proposed suffix that failed or was invalidated by target verification. The loss should therefore place weight on tokens that the draft model itself is likely to propose, especially when those modes are not supported by the target. This leads to the draft-weighted disagreement objective

$$\mathcal{J}_{\text{rej}}(q) = \mathbb{E}_{y \sim q} \left[\log \frac{q(y)}{p(y)} \right]. \quad (17)$$

Since

$$\mathcal{J}_{\text{rej}}(q) = \sum_{y \in \mathcal{V}} q(y) \log \frac{q(y)}{p(y)} = D_{\text{KL}}(q \| p), \quad (18)$$

the rejected-position objective corresponds to reverse KL. Unlike the accepted-position objective, this loss is weighted by the draft distribution and therefore directly penalizes high-probability draft modes that disagree with the target distribution.

Aggregating these local objectives over replay positions gives the acceptance-aware form used in Draft-OPD:

$$\sum_{i \in \mathcal{I}_{\text{acc}}} D_{\text{KL}}(p_i \| q_i) + \sum_{i \in \mathcal{I}_{\text{rej}}} w_i D_{\text{KL}}(q_i \| p_i), \quad (19)$$

up to normalization and scalar weights. This decomposition clarifies the role of the two KL directions. Forward KL is appropriate for accepted positions because the supervision is target-weighted on verified states, while reverse KL is appropriate for rejected positions because the supervision is draft-weighted on states that expose draft errors. Applying a single KL direction to all positions ignores this distinction: all-forward KL treats rejected draft errors like reliable accepted states, whereas all-reverse KL treats verified accepted positions like draft-error states. The component ablations in Table 3 are consistent with this local objective view.

C Hyperparameter Sensitivity

We study the sensitivity of Draft-OPD to the rejected-token decay factor γ and the relative rejected-token weight λ_{rej} . In each sweep, we vary one hyperparameter while keeping all other settings fixed and report the average acceptance length τ on four benchmarks.

Decay factor γ . We sweep γ from 0.5 to 0.95. As shown in Table 8, the average acceptance length remains stable between 6.24 and 6.26 across the entire sweep. In particular, values between 0.5 and 0.8 achieve comparable performance, indicating that Draft-OPD is not sensitive to the precise decay factor.

Table 8: Sensitivity to the rejected-token decay factor γ . All entries report average acceptance length τ .

γ	0.95	0.80	0.65	0.50
AIME25	6.30	6.32	6.34	6.31
MBPP	5.69	5.73	5.71	5.70
MATH-500	6.58	6.57	6.56	6.54
GSM8K	6.37	6.39	6.41	6.39
Average	6.24	6.25	6.26	6.24

Relative rejected-token weight. We fix $\lambda_{\text{acc}} = 1$ and vary λ_{rej} . Table 9 shows that the strongest region is $\lambda_{\text{rej}} \in [0.5, 1]$, with $\lambda_{\text{rej}} = 0.8$ attaining the highest average in this sweep. Larger weights that overemphasize rejected tokens are modestly worse. Overall, performance is stable when the rejected-token weight does not exceed the accepted-token weight, and the default $\lambda_{\text{rej}} = 1$ used in our main experiments lies within this robust range.

Table 9: Sensitivity to the relative rejected-token weight with $\lambda_{\text{acc}} = 1$. All entries report average acceptance length τ .

λ_{rej}	2.0	1.5	1.0	0.8	0.5
AIME25	6.25	6.27	6.32	6.35	6.30
MBPP	5.69	5.72	5.73	5.79	5.71
MATH-500	6.51	6.53	6.57	6.55	6.52
GSM8K	6.34	6.38	6.39	6.37	6.41
Average	6.20	6.23	6.25	6.27	6.24

D Replay Distribution Analysis

We analyze how the verification outcomes associated with replay anchors evolve during OPD training. For each replayed draft block of size $K = 16$, we record the position of the first rejected token within the block. The *Full accept* category denotes that all 16 draft tokens are accepted. Table 10 compares the resulting distribution over the first and last halves of OPD training.

The first-rejection distribution shifts toward later positions during OPD. The probability mass at positions 1–3 decreases from 39.61% to 38.23%, whereas the mass at positions 7–15 increases from 28.48% to 29.50%. The full-accept rate also rises from 5.68% to 6.02%. Since a later first rejection corresponds to more accepted draft tokens within a block, this rightward shift reflects improved draft–target alignment and acceptance over the course of OPD training.

Table 10: Distribution of the first-rejection position within replayed draft blocks during the first and last halves of OPD training.

Position	First 50%	Last 50%	Position	First 50%	Last 50%
1	12.61%	11.98%	9	4.03%	4.18%
2	14.34%	13.86%	10	3.37%	3.50%
3	12.65%	12.38%	11	2.81%	2.92%
4	10.53%	10.44%	12	2.37%	2.47%
5	8.61%	8.63%	13	1.99%	2.09%
6	7.09%	7.19%	14	1.70%	1.78%
7	5.85%	5.99%	15	1.49%	1.55%
8	4.87%	5.03%	Full accept	5.68%	6.02%

E Thinking-Mode Drafting Gap

To examine this gap, we run Qwen3-4B in both thinking and non-thinking modes on prompts from ShareGPT (ShareGPT, 2023), AoPS (Art of Problem Solving, 2026), the math and code splits of Nemotron-Post-Training-Dataset-v2 (Nathawani et al., 2025), and compute the next-token NLL at each token position for the generated responses. Figure 5 shows that thinking-mode responses have higher next-token NLL than non-thinking responses across the evaluated datasets. This suggests that, in thinking mode, the target model itself has higher uncertainty over the next token. A draft model trained for thinking-mode decoding is therefore asked to predict a less concentrated target distribution, making it harder to match the target model’s subsequent tokens accurately.

Draft-OPD partially mitigates this challenge by post-training the draft model on verification-time errors, improving acceptance for reasoning-oriented decoding; however, since our method is designed as a general post-training framework across decoding settings, approaches tailored specifically to reasoning draft models remain an important direction for future work.

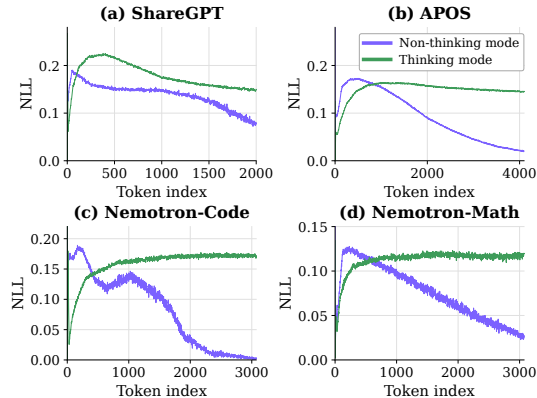


Figure 5: Token-level negative log-likelihood under thinking and non-thinking modes across evaluated datasets.